CHAPTER 1

DESIGNING FACEBOOK

1 Getting Ready: The Facebook System 2
2 Requirements for Facebook 3
3 Use Case Diagram for Facebook . . . 5
4 Class Diagram for Facebook 7
5 Sequence Diagram for Facebook . . . 22
6 Activity Diagram for Facebook . . . 24
7 Code for Facebook 26
8 Summary: Designing Facebook . . . 35
9 Key Insights: Facebook 38

This chapter shows the design of the Facebook system, detailing requirements, use case, class, sequence, activity diagrams, and implementation code.

1 Getting Ready: The Facebook System

1.1 Problem definition

Facebook is a social media platform for users to connect with people and engage with different types of media and content. Users can connect with others by sending friend requests or using the direct messaging feature. Each user has a profile where they can create posts to share with their friends. Facebook also allows its users to create pages regarding a topic of interest and groups to form a community of similar people. Facebook generates a personalized feed for its users based on their friends, liked pages, groups, and the content they engage with to ensure the best experience.

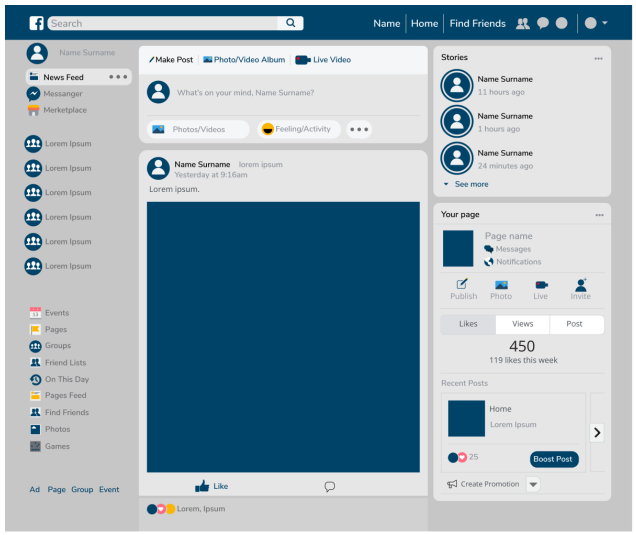


Figure 1.1: The Facebook home page of a user

1.2 Expectations from the interviewee

Facebook offers multiple functionalities to its users. It is important to narrow down what components you will include in your Facebook design. The section below provides an overview of some of the main expectations the interviewer will want to hear you discuss in more detail.

Discoverability

Developing a better understanding of how Facebook's discoverability works is important. The interviewer would expect you to ask the questions listed below:

- How are users able to search other users' profiles?
- Can users search other users using their phone numbers and email addresses?

Friends and following

The "Add friend" feature of Facebook is its most renowned feature. Make sure to ask the interviewer the questions provided below:

- How are users able to add other users as friends?
- How can users follow or unfollow other users without becoming a direct connection?

Groups and pages

Groups and pages on Facebook create a space for like-minded individuals. Make sure to define your requirements. You may ask the interviewer the questions listed below:

- How can users create groups and pages besides their user profiles?
- What are the criteria for joining a group? Can a user join a group if they are not friends with the creator of the group?

Privacy

You can ask the following questions to learn about the privacy feature in your design:

- How will the system handle privacy lists with certain content displayed only to specified connections?
- How do users set the privacy of groups? How does this change the way members and non-members interact with the group?

Alerts

Notifications and alerts allow users to stay updated with the activity in their circle. Therefore, you may want to understand how alerts will work in your system. You may ask the questions listed below:

- How will users be notified of pre-selected events?
- How much control do users have in choosing what notifications they can receive?

1.3 Design approach

We'll design Facebook using the bottom-up design approach. For this purpose, we will follow the steps below:

- Identify and design the smallest components, like a post and comment, first.
- Use these small components to design bigger components, for example, pages, groups, and profiles.
- Repeat the steps above until we design the complete Facebook platform.

1.4 Design pattern

During an interview, it is always a good practice to discuss the design patterns that Facebook follows. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:
Which design pattern(s) should be used to design the Facebook system? Please elaborate on your choice(s).
Let's explore the requirements of Facebook in the next lesson.

2 Requirements for Facebook

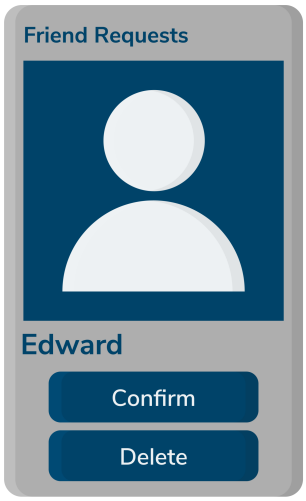
In this lesson, we'll list the requirements of Facebook. This is a very crucial step, as requirements define the scope of a problem, so getting them right from the interviewer and understanding them well will make the design of the rest of the system smooth and easy.
We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for requirement and "n" is a natural number.

2.1 Requirements collection

The requirements for Facebook are defined below:

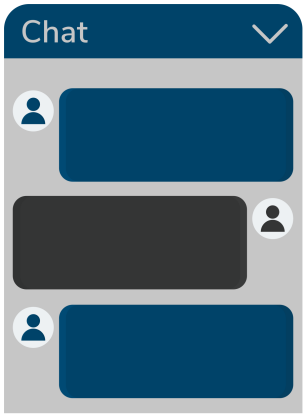
R1: Users should be able to create and edit their profile page by adding personal details such as name, address, phone number, and email. They should also be able to add multiple entries for work experience, education, and places of living. Users should also be able to set the privacy of their profile page.
R2: Our system's users should be able to search for groups, pages, and other users.

R3: Users should be able to write a new post and set its privacy.
R4: Users should be able to send and respond to other users' friend requests. Users should also be able to unfriend or block other users.
R5: Users should be able to follow other users without adding them to their friend list.



R6: Users should be able to like, share, and comment on posts and existing comments.
R7: The system should notify the user whenever there has been an interaction with them, such as receiving a message, a friend request, or a comment on their post.

R8: A user should be able to send and receive messages from other users.
R9: Users should be able to follow existing pages and join existing groups. They should also be able to unfollow or leave joined groups or followed pages.
R10: Users should be able to create their groups and pages. Users can later set privacy or delete the groups or pages they own.



We've identified our requirements for the problem, and in the next lesson, we will define different use cases of Facebook.

3 Use Case Diagram for Facebook

Let's begin building Facebook's use case diagram and understanding its components' relationship. First , we will define the different elements of our Facebook system, followed by the complete use case diagram.

3.1 System

Our system is Facebook.

3.2 Actors

Now, we'll define the main actors of Facebook.

Primary actors

- **User:** This actor can create a personal profile that includes their information. They can create posts, pages, and groups, and interact with other users by sending friend requests and messages, commenting on posts, and more.

Secondary actors

- **Page/Group admin:** The admin is in charge of performing numerous operations, including blocking or unblocking users from groups or pages, deleting an existing group, and changing the group's privacy.
- **System:** This is responsible for sending notifications for new friend requests, messages, comments, etc.

3.3 Use cases

In this section, we'll define Facebook's use cases. We have listed them according to their interactions with a particular actor.

> **Note:** Some use cases will occur multiple times because they are shared among different actors in the system.

User

- **Add/update profile:** To add information like work, education, and places of visit details, or to update an existing profile.
- **Follow/unfollow user:** To follow or unfollow other users.
- **Send message:** To send a message to other users.
- **Send friend request:** To send a friend request to other users.
- **Create/like/follow/share page:** To create a new page or perform actions including liking, following, or sharing an existing page.
- **Create/join/leave group:** To create a new group or perform actions like joining or leaving an existing one.
- **Invite users to group:** To invite other users to an existing group.
- **Add/update/delete post:** To add a new post, update the post's content, or delete an existing post.
- **Like/comment/share post**: To like a post, comment on a post, or share a post.
- **Add/update/delete/like comment:** To add a new comment, update the comment's content, like a particular comment, or delete a comment.

- **Accept/reject friend request:** To accept or reject a friend request from another user.
- **Update privacy:** To update the privacy settings of the profile.
- **Search users/groups/pages/posts:** To search for other users, any existing groups or pages, or any posts made by users.
- **Accept/Reject friend request:** To accept or reject a friend request from another Facebook user.
- **Accept/Reject group join invitation:** To accept or reject a group joining invitation from another Facebook user.
- **Like/Comment/Share post:** To like, comment, or share any post visible to users on Facebook.

Page/Group admin

- **Block/unblock page user:** To block or unblock a user from a page.
- **Block/unblock group user:** To block or unblock a user from a group.
- **Change a group's privacy:** To change the privacy settings of a group (from public to private and vice versa).

System

- **Send new friend request notification:** To send a notification of any friend request sent by another user.
- **Send message notification:** To send a notification of any new messages.
- **Send new post notification:** To send a notification of any new posts.
- **Send comment notification:** To send a notification of any comments on your or others' posts.

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Generalization

- The "Page/Group admin" has a generalization relationship with the "User" as the admin can perform all the tasks a normal user can perform.

Associations

The table below shows the association relationship between actors and their use cases.

Include

- When a user takes an action that triggers a notification to others—such as sending a friend request, message, posting, or commenting—the system notifies the relevant users. These are best represented using "include" relationships:
  - The **Send friend request**, **Send message**, **Add post**, and **Add comment** use cases all include their respective notification use cases:
    - The **Send friend request** use case includes the **Send new friend request notification** use case.
    - The **Send message** use case includes the **Send message notification** use case.

User	Page/Group Admin	System
Add/update profile	Block/unblock page user	Send new friend request notification
Follow/unfollow user	Enable/disable group user	Send message notification
Send message	Change a group's privacy	Send new post notification
Send friend request		Send comment notification
Create/like/follow/share page		
Create/join/leave group		
Invite user to group		
Add/update/delete post		
Add/update/delete/like comment		
Accept/reject friend request		
Update privacy		
Search users/groups/pages/posts		
Accept/Reject friend request		
Accept/Reject group join invitation		
Like/Comment/Share post		

    * The **Add post** use case includes the **Send new post notification** use case.
    * The **Add comment** use case includes the **Send new comment notification** use case.

- When a user updates their profile, they may modify work experience, education, or place of residence.
  - The **Update profile** use case includes the **Add/update work**, **Add/update education**, and **Add/update place** use cases.
- When a user performs a search, the system performs sub-searches across multiple categories.
  - The **Search** use case includes the **Search Users**, **Search Groups**, **Search Pages**, and **Search Posts** use cases.

3.5 Use case diagram

Here's the use case diagram of Facebook:

In the next lesson, we will discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for Facebook

In this lesson, we'll identify and design the classes, abstract classes, and interfaces based on the requirements we previously gathered from the interviewer in our Facebook social network system.

4.1 Components of Facebook

As mentioned, we will design the Facebook social network using a bottom-up approach.

Person

The `Person` class contains details derived from the `User` class, such as name, address, phone number, and email.

Figure 1.2: The use case diagram of Facebook

User The `User` class is Facebook's main class responsible for various operations, such as sending messages or friend requests to other users, creating or joining groups or pages, and numerous others.
The class diagram of these two classes is provided below:

> **Note:** The `User` class contains various methods representing user interactions and functionalities within the system. For brevity, only a subset of these methods is shown here.

R4: Users should be able to send and respond to friend requests, unfriend, and block other users.
R8: A user should be able to send and receive messages from other users.
R9: Users should be able to follow existing pages and join existing groups. They should also be able to unfollow or leave joined groups or followed pages.

Profile

The `Profile` class contains a Facebook user's personal information, such as their profile and cover pictures, work and education details, and a list of places they've visited.
The visual representation of the `Profile` class is given below:

R1: Users should be able to set the privacy of their profile page. They should also be able to create their profile page and add information such as work experience, education, and living place.

Work, education, and places

The `Work`, `Education`, and `Places` classes are used to provide relevant information and are involved in the `Profile` class.
The class representation of these classes is given below:

R1: Users should be able to set the privacy of their profile page. They should also be able to create their profile page and add information such as work experience, education, and living place.

Page

The `Page` class represents a page on the Facebook platform. It contains the name, description, likes count, and a particular ID to uniquely identify the page.
The UML representation of the class is shown below:

R9: Users should be able to follow existing pages and join existing groups. They should also be able to unfollow or leave joined groups or followed pages.

Group

The `Group` class represents a particular group on the Facebook platform. It will contain the name, description, total users, and a unique ID to uniquely identify the group. In addition, it will have methods for adding new users and updating the group's description.
The UML representation of the class is shown below:

R9: Users should be able to follow existing pages and join existing groups. They should also be able to unfollow or leave joined groups or followed pages.

In the Facebook design problem, the `Group` class implements the `GroupFunctions` interface.

Post

The `Post` class indicates a post by any user that contains its content, the number of likes and shares it has, and its owner. Each post has a specific setting as to who can view it.
The class diagram of the `Post` class is shown below:

R3: Users should be able to write a new post and set its privacy.

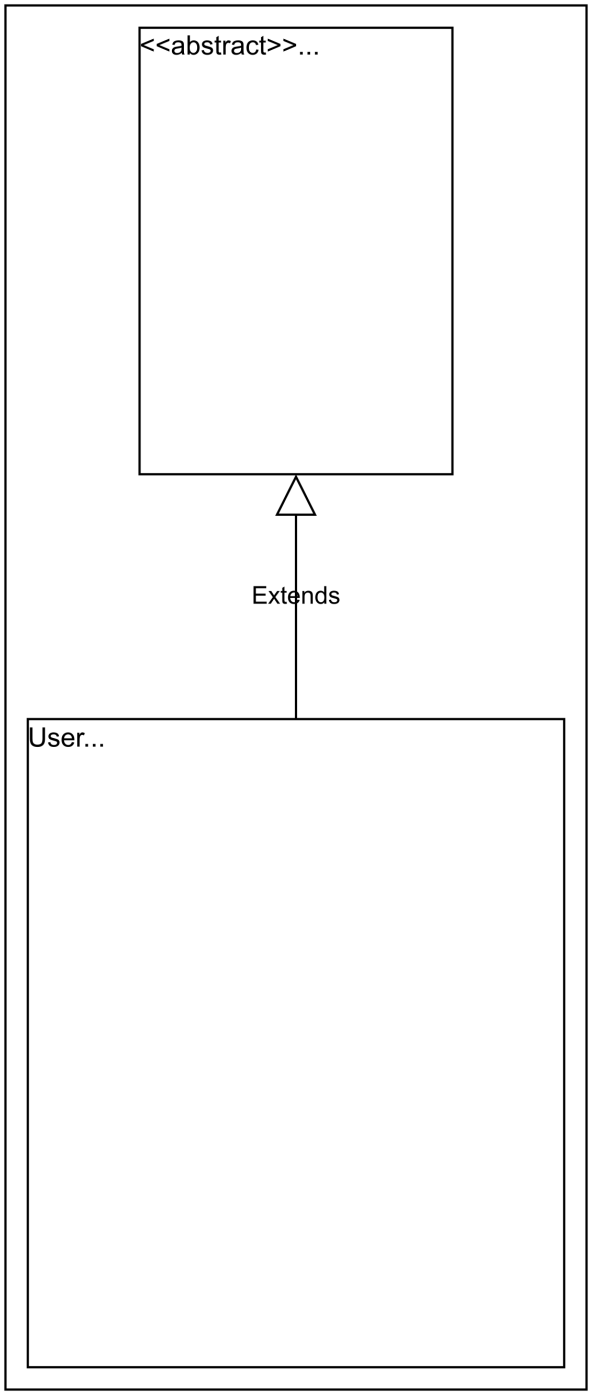


Figure 1.3: The class diagram of the Person and User classes

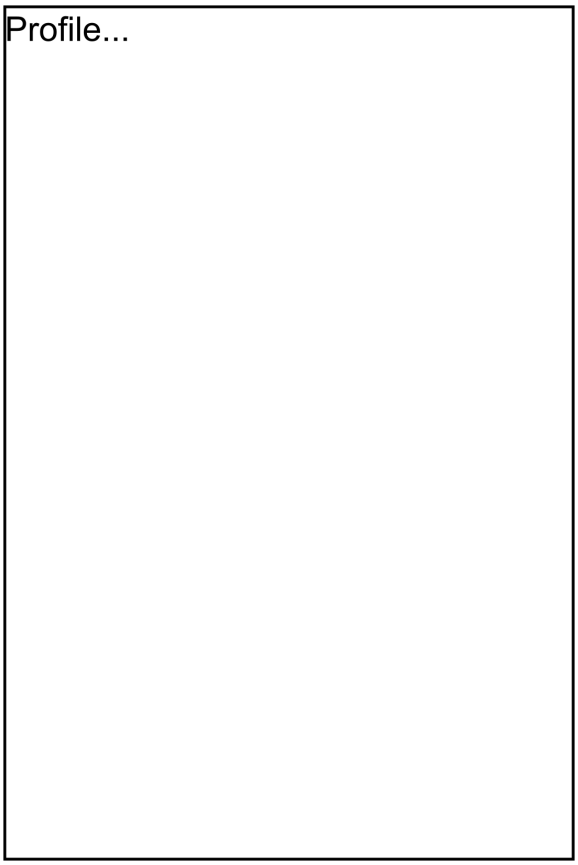


Figure 1.4: The class diagram of the Profile class

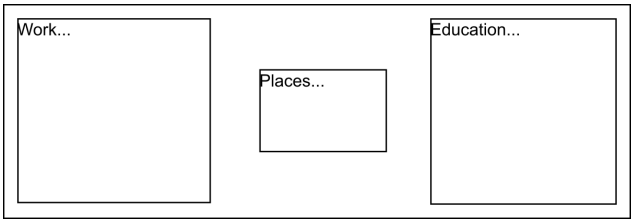


Figure 1.5: The class diagram of the Work, Education, and Places classes

Figure 1.6: The class diagram of the Page class

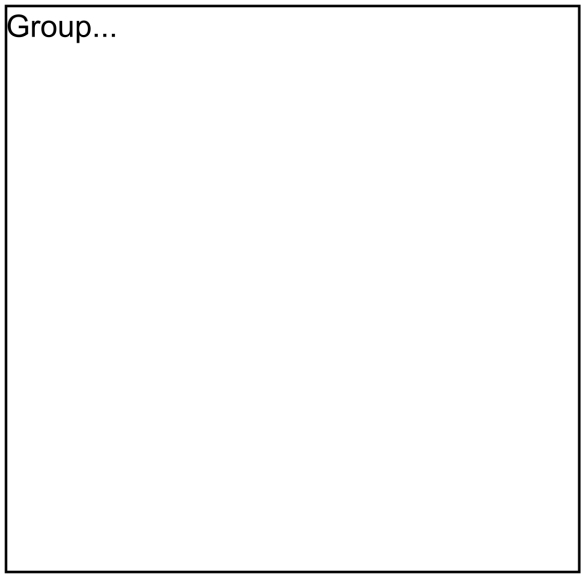


Figure 1.7: The class diagram of the Group class

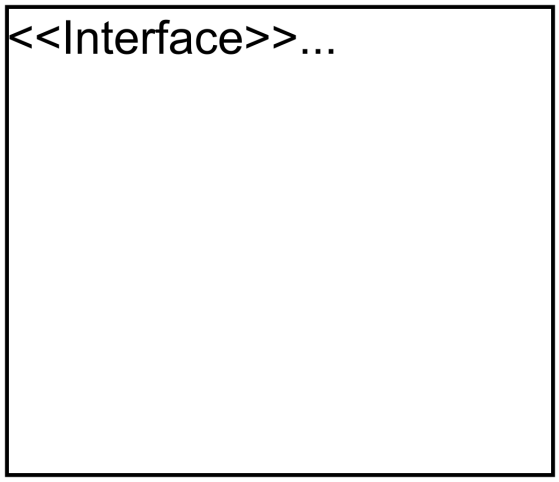


Figure 1.8: The GroupFunctions interface as a subscriber

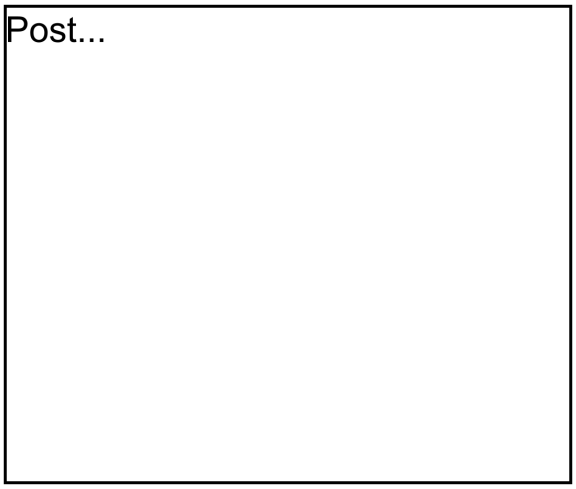


Figure 1.9: The class diagram of the Post class

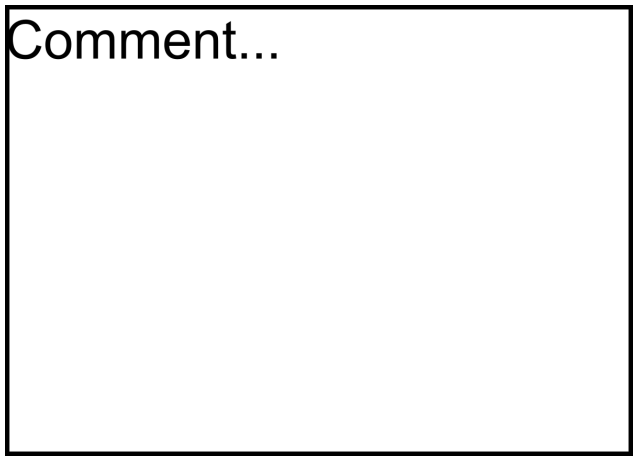


Figure 1.10: The class diagram of the Comment class

Comment

The `Comment` class indicates a comment by any user on a post that will contain its content, the number of likes, and the owner of the comment.
The class diagram of the `Comment` class is shown below:

R6: Users should be able to like, share, and/or comment on a post and like or comment on an existing comment.

Friend request

The `FriendRequest` class describes the details of a friend invitation. It will contain the status of the friend invite and the methods that indicate whether the request was accepted or rejected.
The class diagram of the `FriendRequest` class is shown below:

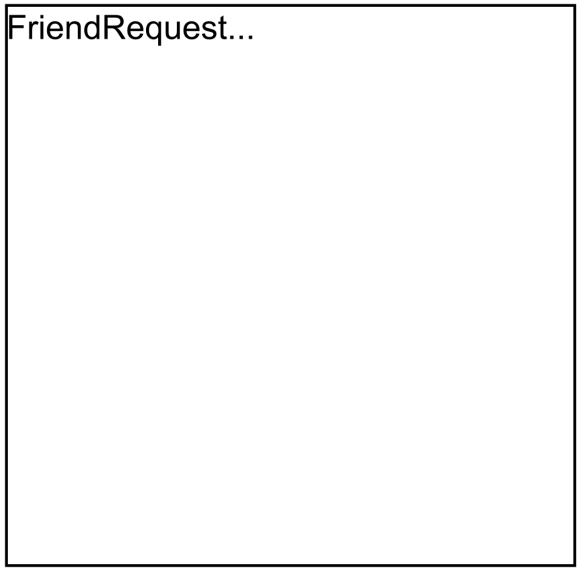


Figure 1.11: The class diagram of the FriendRequest class

R4: Users should be able to send and respond to other users' friend requests. Users should also be able

to unfriend or block other users.

Message

The `Message` class represents the message sent by a user to other users.
The class diagram for the `Message` class is shown below:

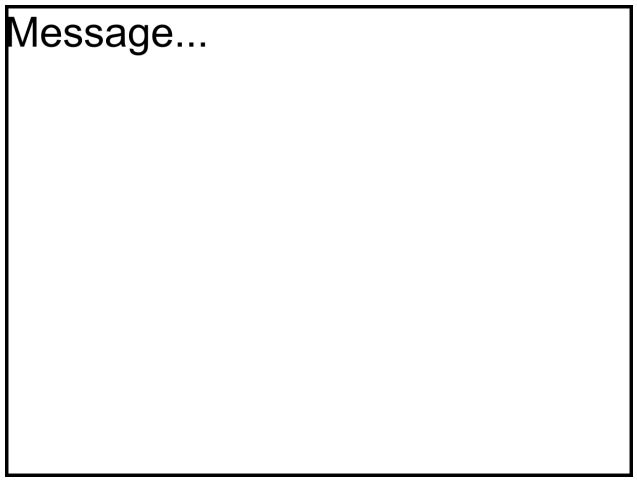


Figure 1.12: The class diagram of the Message class

R8: A user should be able to send and receive messages from other users.

Profile privacy

There will be a privacy class for the profile on Facebook. The following represents the class diagram:

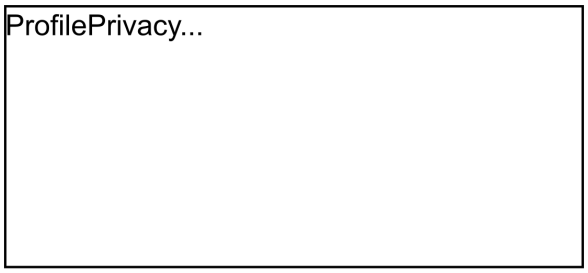


Figure 1.13: The class diagram of the ProfilePrivacy class

R1: Users should be able to set the privacy of their profile page. They should also be able to create their profile page and add information such as work experience, education, and living place.
R3: Users should be able to write a new post and set its privacy.

Notification

The `Notification` class will be a class, since it can send a notification using the built-in notification option. It is mainly responsible for sending notifications whenever any of the following conditions are met:

- A friend request is received.
- A new message is received.

- A new comment is added to a post a user has created or is following.
- A user has created a new post or a page that another user follows.

The UML representation of the class is shown below:

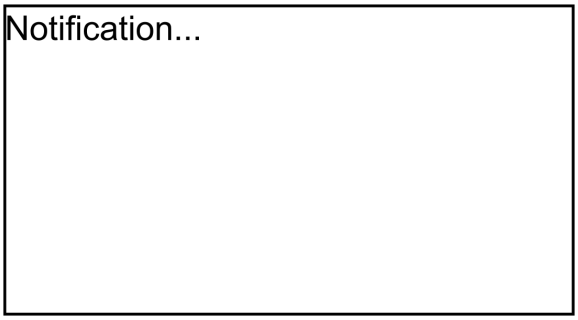


Figure 1.14: The class diagram of the Notification class

R7: The system should notify the user whenever there has been an interaction with them, such as receiving a message, a friend request, or a comment on their post.

Interfaces implemented by the user

Several interfaces will exist in the `Facebook` class, and each will play a separate role in implementing different functionality. The following provides an overview of these:

- `PageFunctionsByUser`**:** This interface will contain the functions that a user will perform while interacting with pages.
- `GroupFunctionsByUser`**:** This interface will contain the user's functions while interacting with groups.
- `PostFunctionsByUser`**:** This interface will contain the functions that a user will perform while interacting with posts.
- `CommentFunctionsByUser`**:** This interface will contain the user's functions while interacting with comments.
- `Search`**:** This interface allows customers to search for any particular user, group, page, or post and returns a list of its respective components.

The UML representation of the interfaces is shown below:

Figure 1.15: The class diagram of interfaces to be implemented by users in Facebook

Search catalog

`SearchCatalog` is the class where the search functionality is implemented. Each catalog contains a list of users, groups, pages, and posts, and users can search for users, groups, pages, and posts.
The class representation of the `SearchCatalog` class is shown below:

R2: Our system's users should be able to search for groups, pages, and other users.

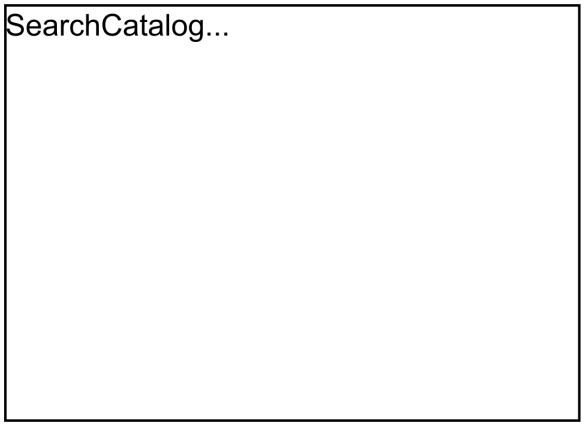


Figure 1.16: The class diagram of the SearchCatalog class

Enumerations

The enumerations required in the Facebook design are listed below:

- `AccountStatus`**:** The account status tells about a member's account status, whether it is active, disabled, or blocked.
- `FriendInviteStatus`**:** The friend invite status describes the status of a friend request, whether it is pending, accepted, rejected, or canceled.
- `PostPrivacySettings`**:** The post-privacy settings enumeration outlines the audience that can view the posts.
- `Role`: The role status describes whether the user is a regular user or has administrative permissions.

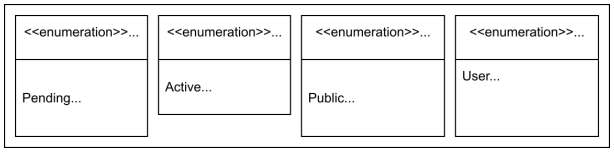


Figure 1.17: Enums in Facebook

Custom data type

We need to create a custom data type, `Address`, to store a Facebook user's location.

4.2 Relationship between the classes

Now, we'll discuss the relationships between the classes we have defined above in our Facebook social network system.

Association

The class diagram has the following association relationships:

- The `User` class has a one-way association with:

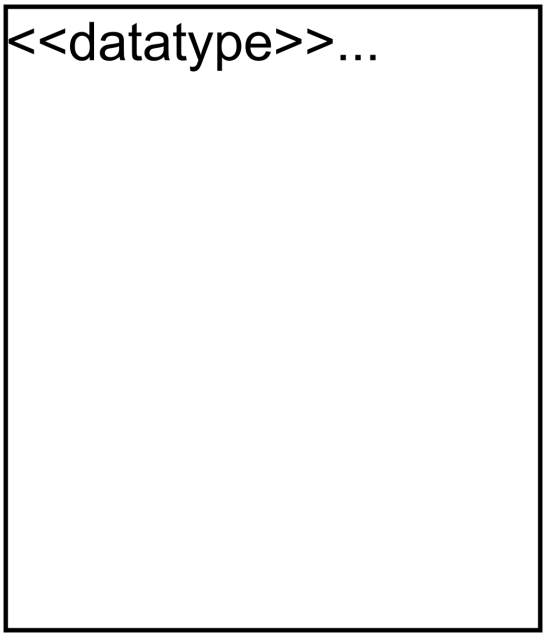


Figure 1.18: The class diagram of the custom Address data type

  - Itself
  - The `Page`, `Group`, `FriendRequest`, `Message`, `Comment`, `Post`, `ProfilePrivacy`, and `PostPrivacy` classes.
  - The `PageFunctionsByUser`, `GroupFunctionsByUser`, `CommentFunctionsByUser`, `PostFunctionsByUser`, and `Search` interfaces.

- The `Profile` class has a one-way association with `ProfilePrivacy`.
- The `Post` class has a one-way association with `PostPrivacy` and `Notification`.
- The `Notification` class has a one-way association with `FriendRequest`, `Message`, and `Comment`.

Composition

The class diagram has the following composition relationships:

- The `User` class is composed of the `Profile` class.

Aggregation

The following classes show an aggregation relationship:

- The `Profile` class contains the `Work`, `Education`, and `Places` classes.

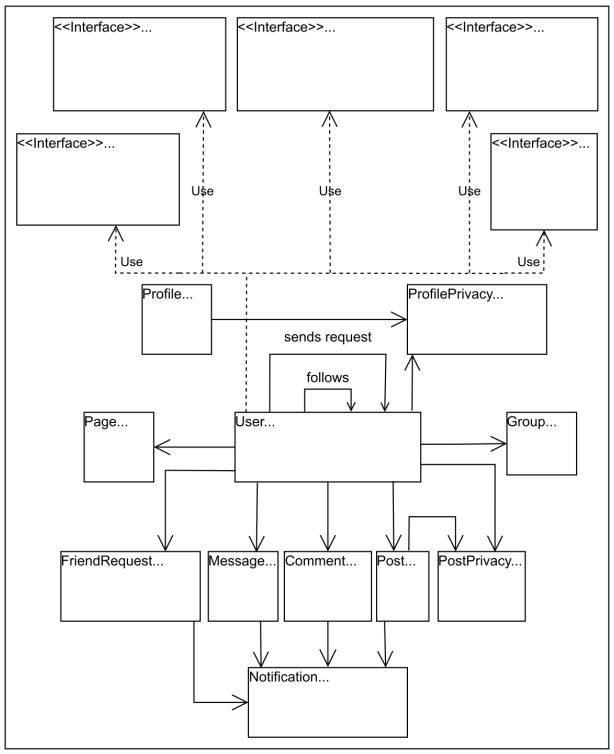


Figure 1.19: The association relationship between classes

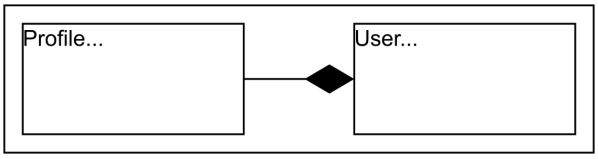


Figure 1.20: The composition relationship between classes

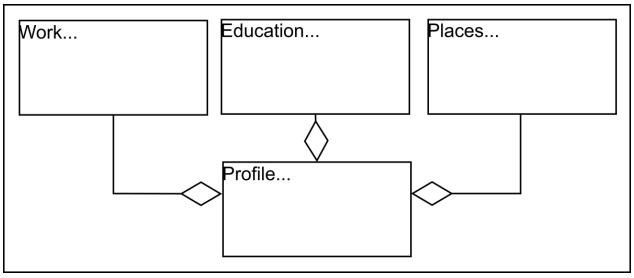


Figure 1.21: The aggregation relationship between classes

Generalization

The following classes show a generalization relationship:

- The `SearchCatalog` class implements the `Search` interface.
- The `Group` class implements the `GroupFunctions` interface.

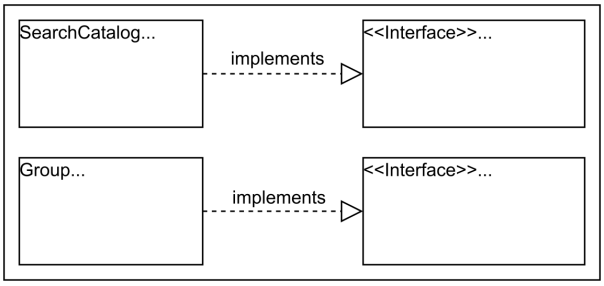


Figure 1.22: The generalization relationship between classes

Inheritance

The following classes show an inheritance relationship:

- The `User` class extends the `Person` class.

4.3 Class diagram of Facebook

This section outlines the multiplicity (cardinality) relationships between the main classes in our Facebook design system. We explain the allowed number of instances on each side for each relationship and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Here's the complete class diagram for Facebook:

4.4 Design pattern

We apply object-oriented design patterns based on system behavior to implement Facebook's core features flexibly and scalable.

- We know that Facebook notifies users of new posts, comments, or messages. To model this behavior, we can use the Observer design pattern.
- We know that Facebook allows users to choose different privacy settings for posts and profiles. To handle this, we can use the Strategy design pattern.
- Facebook sends different types of notifications. The Factory Method design pattern allows us to create these flexibly.
- We know that Facebook uses a global search feature. We can use the Singleton design pattern to ensure a single shared search service.
- Facebook users interact with many features, such as posts, groups, and pages. The Interface Segregation principle can separate these responsibilities.
- We know that Facebook handles status changes in friend requests and user accounts. To manage state-specific behavior, we can use the State design pattern.

Source	Target	Multiplicity	Reason
User	Post	1 – 0..*	A user can create, like, share, and comment on multiple posts.
Post	User	1 – 1	Each post is associated with exactly one user.
User	Comment	1 – 0..*	A user can create and interact with multiple comments.
User	Message	1 – 0..*	A user can create and interact with multiple messages.
User	FriendRequest	1 – 0..*	A user can create and interact with multiple friend requests.
User	Notification	1 – 0..*	A user can receive multiple notifications.
FriendRequest	User	1 – 2	Friend requests involve two users (sender and recipient).
User	Page	1 – 0..*	A user can follow, like, or share multiple pages.
User	Group	1 – 0..*	A user can join or manage several groups.
User	Profile	1 – 1	Each user is associated with exactly one profile.
Profile	Work	1 – 0..*	A profile may include multiple work entries.
Profile	Education	1 – 0..*	A profile may include multiple education entries.
Profile	Places	1 – 0..*	A profile may include multiple place entries.
Profile	ProfilePrivacy	0..1 – 1	Profile may optionally be linked to a privacy setting.
Group	User	1 – 1..*	A group consists of one or more users as members.
Page	User	1 – 1..*	At least one user manages a page.
User	Group, Page	1 – 0..*	Users can participate in multiple groups and pages.
SearchCatalog	User, Group, Page, Post	1 – 0..*	Maps searchable terms to multiple corresponding objects.

Table 1.1: Social Media System Entity Relationships

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your

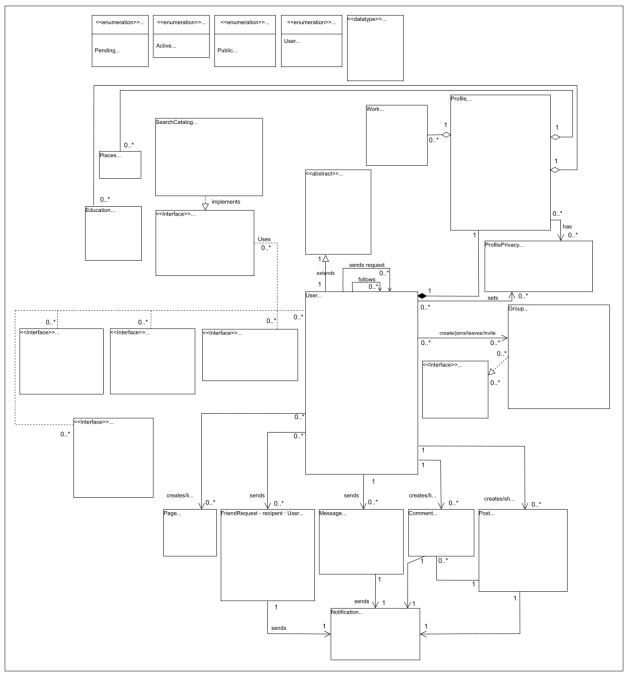
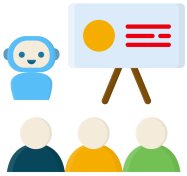


Figure 1.23: The class diagram of Facebook

understanding of the concepts.



Component type 'PromptAI' not yet supported.

4.6 Additional requirements

The interviewer can introduce some additional requirements in the Facebook social network system, or they can ask some follow-up questions. Let 's see some examples of additional requirements:

- `Recommendations:` Users can send recommendations of pages and groups to other users. The class diagram provided below shows the relationship of `Recommendation` with the other classes:

We have completed the Facebook class diagram according to the requirements. In the next lesson, let's design the Facebook sequence diagram.

5 Sequence Diagram for Facebook

Sequence diagrams are a great way to understand the interactions between different entities and objects in the system. There can be different sequence diagrams that we can create for Facebook. In this lesson, we will create sequence diagrams for the following interaction:

- **Send a friend request:** A user sends a friend request to another user.

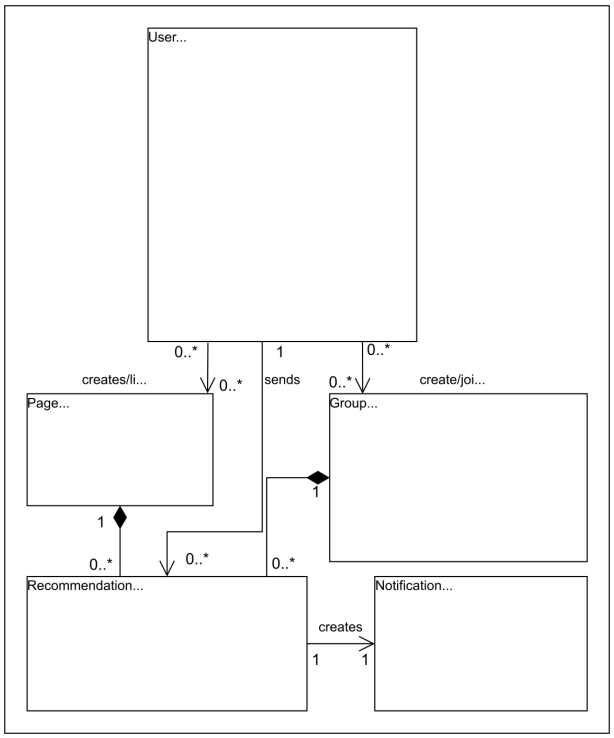


Figure 1.24: The class diagram of the Recommendation functionality

5.1 Send a friend request

The sequence diagram for sending a friend request should have the following actors and objects that will interact with each other:

- **Actors:** User A and User B
- **Objects:** `Catalog` and `FriendRequest`

Here are the steps for the "send a friend request" interaction:

1. User A searches for user B in the catalog.
2. If User B exists:
   (a) The catalog returns user B
   (b) User A adds user B as a friend.
   (c) The friend request is sent to user B.
   (d) If the request is accepted:
      i. User B is added to user A's friend list.
      ii. User A is notified that the friend request has been accepted.
   (e) Otherwise, the request is rejected.
3. Else, if user B does not exist:
   (a) User A receives a "user not found" error.

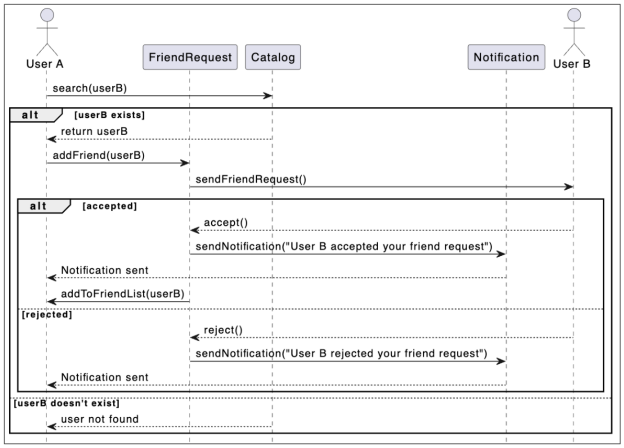


Figure 1.25: The sequence diagram for sending a friend request

Based on the order above, the sequence diagram for sending a friend request on Facebook is given below:

Next, let's look at the activity diagrams for Facebook to understand the control flow of the system.

6 Activity Diagram for Facebook

Activity diagrams are a great way to visualize the flow of messages from one activity to another in the system. There can be different activity diagrams that we can create for Facebook. In this lesson, we will create activity diagrams for the following two activities:

- Creating a new post.
- **Activity challenge:** A user joins a group.

6.1 Creating a new post

The states and actions involved in this activity diagram are provided below.

States

Initial state: The user selects the new post option.
Final state: The post is published.

Actions

The user selects the new post option, selects the privacy option, adds any attachments, and publishes the post.

6.2 Activity challenge: A user joins a group

You'll help us create an activity diagram of a user joining a group. A skeleton of the activity diagram is given below:

Notice that the actions in the diagram above are numbered from 1 to 11. The slots in the above diagram represent the activities, and the arrows represent the flow from one activity to the other.
Can you rearrange the slots below into the correct order in which they should appear in the activity diagram above?

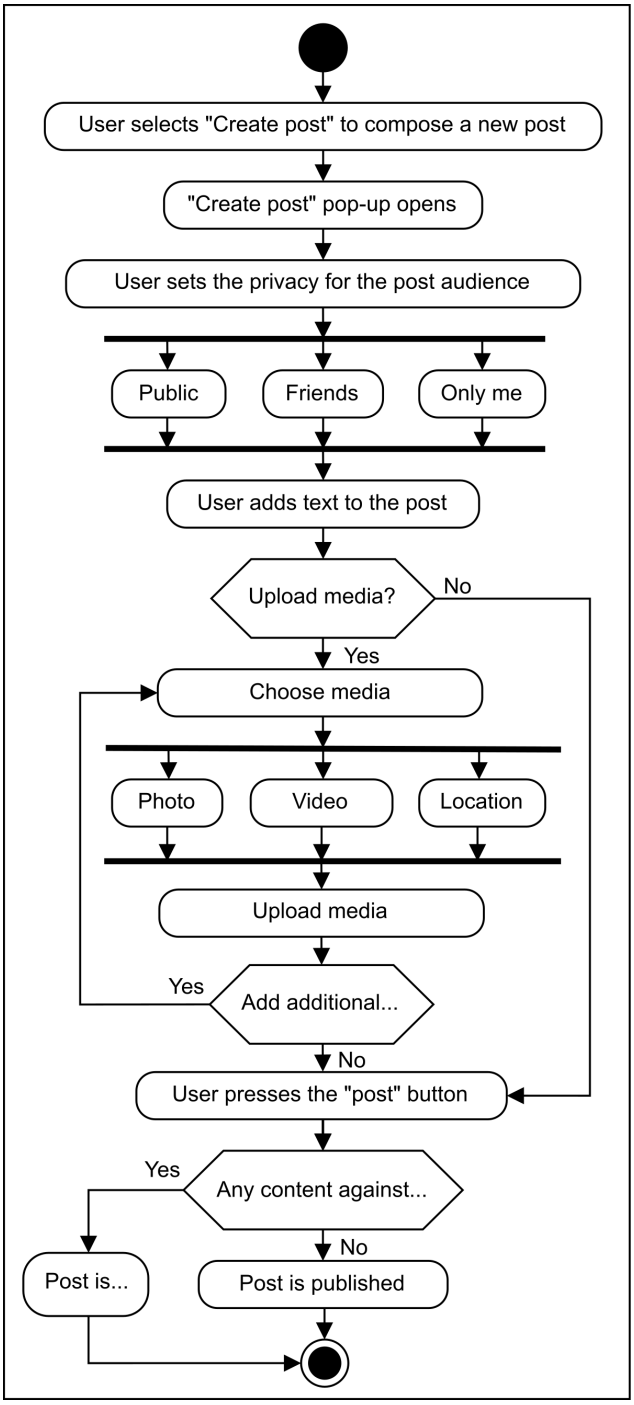


Figure 1.26: The activity diagram for a user creating a new post

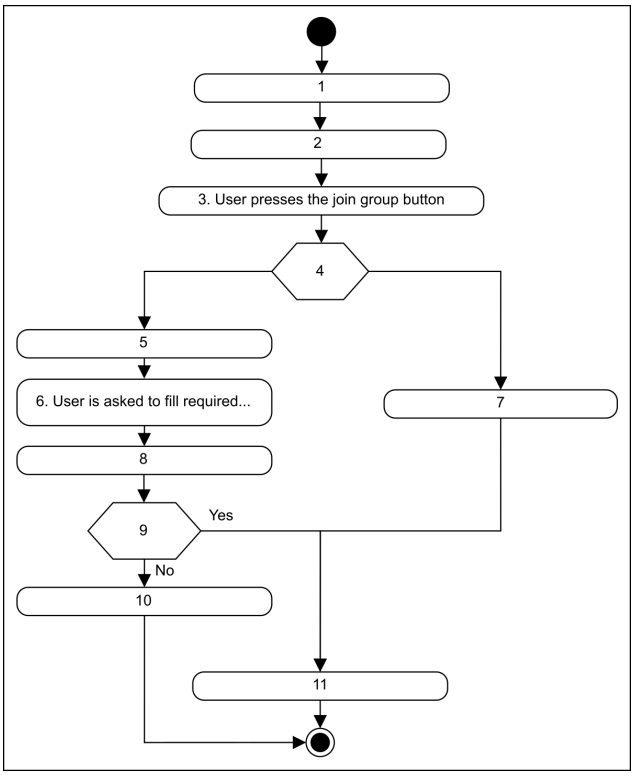


Figure 1.27: The activity diagram for a user joining the group

Note: If you're stuck, click the "Show Solution" button.

Component type 'Permutation' not yet supported.

Alternatively, you can click the "Show complete diagram" button below to see the complete activity diagram.

We've looked at some of Facebook's activity diagrams. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

7 Code for Facebook

We've reviewed different aspects of Facebook and observed the attributes attached to the problem using various UML diagrams. Let 's now explore the more practical side of things, where we will work on implementing the Facebook network using multiple languages. This is usually the last step in an object-oriented design interview process.

We have chosen the following languages to write the skeleton code of the different classes present in Facebook:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click **Executable Code: Designing Facebook**.

7.1 Facebook classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Constants

The following code defines the various enums and custom data types being used in the Facebook design:

> **Note:** JavaScript does not support enumerations, so we will use the Object.freeze() method as an alternative that freezes an object and prevents further modifications.

Constant definitions

```
public class Address {
  private int zipCode;
  private String houseNo;
  private String city;
  private String state;
  private String country;
}

enum AccountStatus {
  ACTIVE,
  BLOCKED,
  DISABLED,
  DELETED
}

enum FriendInviteStatus {
  PENDING,
  ACCEPTED,
  REJECTED,
  CANCELED
}

enum PostPrivacySettings {
  PUBLIC,
  FRIENDS_OF_FRIENDS,
  ONLY_FRIENDS,
  CUSTOM
}

enum Role {
    USER,
    ADMIN
}
```

Interfaces implemented by the user

Facebook will have several interfaces that will be implemented by users and are described below:

- `PageFunctionsByUser`: This will define a user's functions while interacting with pages.
- `GroupFunctionsByUser`: This will define a user's functions while interacting with groups.

- `PostFunctionsByUser`: This will define a user's functions while interacting with posts.
- `CommentFunctionsByUser`: This will define a user's functions while interacting with comments.

The definition of all these interfaces is given below:

The interfaces present in Facebook

```
public interface PageFunctionsByUser {
  public Page createPage(String name);
  public Page sharePage(Page page);
  public void likePage(Page page);
  public void followPage(Page page);
  public void unLikePage(Page page);
  public void unFollowPage(Page page);
}

public interface GroupFunctionsByUser {
  public Group createGroup(String name);
  public void joinGroup(Group group);
  public void leaveGroup(Group group);
  public void sendGroupInvite(Group group);
}

public interface PostFunctionsByUser {
  public Post createPost(String content);
  public Post sharePost(Post post);
  public void commentOnPost(Post post);
  public void likePost(Post post);
}

public interface CommentFunctionsByUser {
  public Comment createComment(Post post, String content);
  public void likeComment(Comment comment);
}
```

Person and user

The `Person` class is an abstract class that serves as the blueprint for all Facebook users. It defines common attributes such as ID, password, name, email, phone number, address, and account status. It also includes a method to reset the user's password.
The `User` class extends `Person` and implements user-specific functionality through the `PageFunctionsByUser` interface. Each `User` object maintains a profile, a list of pages and groups they administer, and a set of roles they hold. By default, every user has the `User` role but can also assume additional roles, such as `Admin`. The definitions of the relevant classes are provided below:

> **Note:** The `User` class contains various methods representing user interactions and functionalities within the system. For brevity, only a subset of these methods is shown here.

The Person, User, and Admin classes

```
public abstract class Person {
  private String Id;
  private String password;
  private String name;
  private String email;
  private String phone;
  private Address address;
  private AccountStatus status;

  public boolean resetPassword();
}
```

```
public class User extends Person implements PageFunctionsByUser{
  private Date dateOfJoining;
  private List<Page> pagesAdmin;
  private List<Group> groupsAdmin;
  private Profile profile;

  private Set<Role> roles = new HashSet<>();

  public boolean hasRole(Role role);

  public boolean sendMessage(Message message);
  public boolean sendRecommendation(Page page, Group group, User user);
  public boolean sendFriendRequest(User user);
  public boolean unfriendUser(User user);
  public boolean blockUser(User user);
  public boolean followUser(User user);

  public Page createPage(String name);
  public void likePage(Page page);
  public void followPage(Page page);
  public void unLikePage(Page page);
  public void unFollowPage(Page page);
  public Page sharePage(Page page);
  public List<User> search(String name);
  public boolean addFriend(User userB);
  public boolean accept(FriendRequest request);
  public boolean reject(FriendRequest request);
  public void enablePage(Page page) {}
  public void disablePage(Page page) {}

  // If user is Admin
  public void blockPageUser(User user) {}
  public void unblockPageUser(User user) {}
  public void blockGroupUser(User user) {}
  public void unblockGroupUser(User user) {}
  public void changeGroupPrivacy(Group group) {}

}
```

Profile, education, places, and work

The `Work`, `Education`, and `Places` classes will provide a user's personal information and make up the `Profile` class. The definition of these classes is given below:

The Profile, Education, Places, and Work classes

```
public class Profile {
  private String gender;
  private byte[] profilePicture;
  private byte[] coverPhoto;
  private List<User> friends;

  private List<int> usersFollowed;
  private List<int> pagesFollowed;
  private List<int> groupsJoined;

  private ProfilePrivacy privacy;

  private List<Work> workExperience;
  private List<Education> educationInfo;
  private List<Place> places;

  public boolean addWorkExperience(Work work);
  public boolean addEducation(Education education);
  public boolean addPlace(Place place);
```

```
  public boolean addProfilePicture(byte[] image);
  public boolean addCoverPhoto(byte[] image);
  public boolean addGender(String gender);
}

public class Work {
  private String title;
  private String company;
  private String location;
  private String description;
  private Date startDate;
  private Date endDate;
}

public class Places {
  private String name;
}

public class Education {
  private String school;
  private String degree;
  private String description;
  private Date startDate;
  private Date endDate;
}
```

Page, post, and comment

Facebook users can create and like pages, posts, and comments. The definition of these classes is given below:

The Page, Post, and Comment classes

```
public class Page {
  private int pageId;
  private String name;
  private String description;
  private int likeCount;
}

public class Post {
  private int postId;
  private String content;
  private byte[] image;
  private int likeCount;
  private int shareCount;
  private User postOwner;
  private PostPrivacySettings settings;
  public changePostVisibility(Post post);
}

public class Comment {
  private int commentId;
  private String content;
  private int likeCount;
  private User commentOwner;
}
```

Profile privacy

Each profile will have privacy settings, where users can change the visibility settings of their friends' list and lock their profile and profile picture.

The ProfilePrivacy class

```
public class ProfilePrivacy {
    public void changeFriendsListVisibility(Profile profile) {}
    public void lockProfile(Profile profile) {}
    public void lockProfilePicture(Profile profile) {}
}
```

Group and the group functions interface

Facebook users can also create and join groups, add and delete new users. In addition, users who initially join groups will be notified of all new activities. The definition of these classes is given below:

The Group class and the GroupFunctions interface

```
public interface GroupFunctions {
  public boolean addUser(User user);
  public boolean deleteUser(User user);
  public boolean notifyUser(User user);
}

public class Group implements GroupFunctions {
  private int groupId;
  private String name;
  private String description;
  private byte[] coverPhoto;
  private int totalUsers;
  private boolean isPrivate;
  private List<User> users;

  public boolean addUser(User user) {
    // functionality
  }
  public boolean deleteUser(User user) {
    // functionality
  }
  public boolean notifyUser(User user) {
    // functionality
  }
  public void updateDescription(String description) {}
  public void addCoverPhoto(byte[] image) {}
}
```

Message and friend request

Each Facebook user can send messages and friend requests to other users. The definition of both of these classes is given below:

The Message and FriendRequest classes

```
public class Message {
  private int messageId;
  private User sender;
  private String content;
  private List<User> recipent;
  private List<byte[]> multimedia;

  public boolean addRecipent(List<User> users);
}

public class FriendRequest {
  private User recipent;
```

```
  private User sender;
  private FriendRequestStatus status;
  private Date requestSent;
  private Date requestStatusModified;

  public boolean acceptRequest(User user);
  public boolean rejectRequest(User user);
  public boolean sendFriendRequest() {
    // Send friend request logic
  }

  public boolean addToFriendList(User userB) {
    // Add userB to sender and recipient's friend list
  }

  public boolean sendNotification() {
    // Notify both parties
  }

}
```

Notification

The `Notification` class sends notifications to users about new messages, comments, posts, or friend requests via the built-in notification option. Its definition is given below:

The Notification class

```
public class Notification {
  private int notificationId;
  private Date createdOn;
  private String content;

  public boolean sendNotification(Profile profile);
}
```

Search catalog and interface

The `SearchCatalog` class contains information on members, groups, pages, and posts. It also implements the `Search` interface class to enable the search functionality based on criteria (member, group, page names, and post keywords). The definition of these two classes is given below:

The Search interface and SearchCatalog class

```
public interface Search {
  public List<User> searchUsers(String name);
  public List<Group> searchGroups(String name);
  public List<Page> searchPages(String name);
  public List<Post> searchPosts(String keywords);
}

public class SearchCatalog implements Search {
  HashMap<String, List<User>> userNames;
  HashMap<String, List<Group>> groupNames;
  HashMap<String, List<Page>> pageTitles;
  HashMap<String, List<Post>> posts;

  public boolean addNewUser(User user) {}
  public boolean addNewGroup(Group group) {}
  public boolean addNewPage(Page page) {}
  public boolean addNewPost(Post post) {}
```

```
  public List<User> searchUsers(String name) {
    // functionality
  }

  public List<Group> searchGroups(String name) {
    // functionality
  }

  public List<Page> searchPages(String name) {
    // functionality
  }

  public List<Post> searchPosts(String keywords) {
    // functionality
  }
}
```

7.2 Executable code: Designing Facebook

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the Facebook system. The main driver code of the system resides in the `Driver.java`, `Driver.cs`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the codes.

What does this code show

- **System initialization:** Initializes two users (`Alice` and `Bob`) with their own `Profile` instances and empty friend lists. This sets up the baseline for simulating user interactions.
- **Scenario 1 (Friend request and acceptance):** Alice searches for Bob and sends him a friend request. Bob accepts the request, and both users' friend lists are updated to reflect the new connection.
- **Scenario 2 (Page creation and following):** Alice creates a new page titled "Tech Updates." Bob follows this page, simulating how users can engage with community or brand content on the platform.
- **Scenario 3 (Group creation and joining):** A group called "Java Enthusiasts" is created directly. Alice joins the group using the `addUser` method, demonstrating user engagement with interest-based communities without altering the core `User` class.

  Hint: Try customizing the code!
  This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Executable Code: Designing Facebook

```
import java.util.*;

public class Driver {
  public static void main(String[] args) {

    System.out.println("========================================");
    System.out.println(" SCENARIO 1: Friend Request and Acceptance");
    System.out.println("========================================");

    // Create Profiles
    Profile profileA = new Profile();
    profileA.setFriends(new ArrayList<>());

    Profile profileB = new Profile();
    profileB.setFriends(new ArrayList<>());
```

```java
    // Create Users
    User userA = new User();
    userA.setName("Alice");
    userA.setProfile(profileA);

    User userB = new User();
    userB.setName("Bob");
    userB.setProfile(profileB);

    // Step 1: UserA searches for UserB
    System.out.println(" UserA searches for UserB...");
    System.out.println(" Found user: " + userB.getName());

    // Step 2: UserA sends a friend request
    FriendRequest friendRequest = new FriendRequest();
    friendRequest.setSender(userA);
    friendRequest.setRecipent(userB);
    friendRequest.sendFriendRequest();
    System.out.println(" Friend request sent from " + userA.getName() + " to " + userB.getName());

    // Step 3: UserB accepts the friend request
    boolean accepted = userB.accept(friendRequest);
    if (accepted) {
      System.out.println(" " + userB.getName() + " accepted the friend request.");
    }

    // Step 4: Display updated friend lists
    System.out.println(" Friends of " + userA.getName() + ": " +
        userA.getProfile().getFriends().stream().map(User::getName).toList());
    System.out.println(" Friends of " + userB.getName() + ": " +
        userB.getProfile().getFriends().stream().map(User::getName).toList());

    System.out.println("
========================================");
    System.out.println(" SCENARIO 2: User Creates and Follows a Page");
    System.out.println("========================================");

    // Step 1: UserA creates a page
    Page techPage = userA.createPage("Tech Updates");
    System.out.println(" " + userA.getName() + " created a page: Tech Updates");

    // Step 2: UserB follows the page
    userB.followPage(techPage);
    System.out.println(" " + userB.getName() + " followed the page: Tech Updates");

    System.out.println("
========================================");
    System.out.println(" SCENARIO 3: Creating and Joining a Group");
    System.out.println("========================================");

    // Step 1: A group is created directly
    Group javaGroup = new Group();
    javaGroup.setName("Java Enthusiasts");
    javaGroup.setUsers(new ArrayList<>());
    javaGroup.setTotalUsers(0);
    javaGroup.setPrivate(false);
    System.out.println(" A new group created: " + javaGroup.getName());

    // Step 2: UserA joins the group
    boolean joined = javaGroup.addUser(userA);
    if (joined) {
      System.out.println(" " + userA.getName() + " joined the group: " + javaGroup.getName());
    } else {
      System.out.println(" " + userA.getName() + " could not join the group (already a member?)");
    }
  }
}
```

7.3 Wrapping up

We've explored the complete design of Facebook in this chapter. We 've explor ed Facebook's complete design, including how it can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing Facebook

Now that you've completed the Facebook case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

8.1 Key requirements

The following are the primary functional and non-functional requirements for the Facebook system:

1. Users can create, edit, and set privacy for their profile, including personal info, work, education, and places.
2. Users can search for other users, groups, pages, and posts.
3. Users can create posts, set privacy for each post, and add attachments.
4. Users can send, accept, reject, or block friend requests, and unfriend users.
5. Users can follow or unfollow other users without being friends.
6. Users can like, share, and comment on posts and comments.
7. The system notifies users about relevant events (messages, friend requests, comments, etc.).
8. Users can send and receive messages.
9. Users can create, join, leave, follow, or unfollow pages and groups, and set group/page privacy or delete them.
10. Page/Group admins can manage members, block/unblock users, and change privacy settings.

8.2 System actors

The following are the main actors who interact with the Facebook system:

- **User:** Creates a profile, connects with others, posts content, joins groups/pages, sends messages, and manages privacy.
- **Page/Group admin:** Manages group/page settings, user membership, and content moderation.

8.3 Important classes and relationships

Here's a summary of the key classes, their responsibilities, and main relationships:

8.4 Design highlights

This section summarizes key design decisions and rationale in Facebook's LLD:

Class	Description	Important Relationships
Person	Stores basic info such as name, address, phone, and email.	Parent of User; uses Address.
User	Represents a Facebook user, handles all major user actions.	Inherits from Person; composed of Profile; associated with Post, Comment, FriendRequest, Message, Notification, Page, Group.
Profile	Holds user's bio, work, education, places, profile/cover pics.	Aggregates Work, Education, Places; associated with ProfilePrivacy.
Page	Represents a Facebook page with name, description, and admins.	Managed by User; associated with Notification.
Group	Represents a Facebook group, includes members, and privacy settings.	Managed by User; implements GroupFunctions interface.
Post	User-generated content includes content, likes, shares, and the owner.	Owned by User; associated with PostPrivacy, Notification, Comment.
Comment	User comment on a post; includes content, likes, and owner.	Owned by User; associated with Post.
FriendRequest	Tracks a friend's invitation status between users.	Involves two User instances; associated with Notification.
Message	Direct messages between users.	Sent/received by User.
Notification	Sends notifications for events (friend requests, messages, comments, etc.).	Associated with User, Post, FriendRequest, Message, Comment.
Search	Interface for searching different Facebook entities.	Implemented by SearchCatalog.
Address	Stores the physical address of a user.	Used by Person.

Table 1.2: Social Media System Classes and Their Relationships

8.5 Design approach

The Facebook system uses a **bottom-up approach**, starting with fundamental units (posts, comments), then composing them into more complex features (profiles, groups, pages), and integrating them into the full platform.

8.6 Design principles

The design demonstrates the SOLID principles:

- **Single Responsibility principle (SRP):** Each class handles one clear aspect (e.g., `Profile` for bio info, `Notification` for alerts).
- **Open/Closed principle (OCP):** By extending classes/interfaces, new features (e.g., new post/privacy types) can be easily added.
- **Liskov Substitution principle (LSP):** Subclasses (`User`, `SearchCatalog`, `Group`) can be used wherever their parent/interface is expected.
- **Interface Segregation principle (ISP):** Distinct interfaces for searching, group/page/post/comment functions keep code modular.

- **Dependency Inversion principle (DIP):** High-level features depend on abstractions (e.g., `Search`, privacy settings) instead of concrete implementations.

8.7 Design patterns

Key patterns in Facebook's system include:

- **Observer pattern:** Used for notifications; users are notified of events like messages, friend requests, and post activity.
- **Singleton pattern:** Ensures the notification system has a single control point for sending user alerts.
- **Factory pattern:** Used to create instances of `User`, `Post`, and `Page` objects, which allows for flexible object creation and management.
- **Strategy pattern:** This pattern enables the dynamic selection of privacy settings or feed ranking strategies for posts and profiles, allowing users and the system to adapt visibility and personalization features flexibly.
- **Interface Segregation and Abstraction:** The system uses distinct interfaces (such as `Search`, `PageFunctionsByUser`, `GroupFunctionsByUser`, `PostFunctionsByUser`, and `CommentFunctionsByUser`) to separate different user and admin actions. This keeps the code modular, focused, and easy to extend with new features.

8.8 Object interactions

Here's how the main objects collaborate dynamically:
Sending a friend request:

- User A searches for User B via `SearchCatalog`.
- User A sends a friend request to User B using `FriendRequest` if found.
- The system sends a notification to User B.
- User B accepts or rejects the request; both users become friends upon acceptance, and User A is notified.

Creating a post:

- User selects the "new post" option and enters content.
- The user sets privacy options and publishes the post.
- System notifies relevant users/followers; the post appears in friends' feeds.

Joining a group:

- User searches for a group, requests to join.
- If public, the user joins directly; if private, the request goes to the group admin for approval.
- Upon approval, the user is added as a member and notified.

Messaging:

- User sends a message; recipient receives a notification.

8.9 System workflow

This section outlines the main end-to-end scenarios for Facebook users:

- **User registration and profile setup:** A new user enters personal details like name, address, phone number, and email, then adds work experience, education, and places lived. Users can configure profile privacy settings to control who can view their information.
- **Content discovery and engagement:** Users use the search feature to find other users, groups, pages, or posts on the platform. They can also follow users, join groups, or follow pages and interact by liking, commenting, or sharing posts and comments.
- **Building connections:** Users connect with others by sending friend requests, which can be accepted or rejected. They can also follow users without sending a friend request to see their public activities, and unfriend or block others if needed.
- **Content creation and sharing:** Users create posts and set privacy preferences for each one. Shared posts appear in the feeds of friends or followers, who can then like, comment on, or share them.
- **Group/page interactions:** Users can create groups or pages, join or follow existing ones, and set privacy options. Groups and pages enable users to connect with communities and discuss shared interests.
- **Notifications and messaging:** The system automatically sends notifications for friend requests, messages, new posts, and comments. Users can send and receive direct messages to communicate privately with other users.
- **Privacy and security:** Users manage privacy settings for their profiles, posts, and groups, deciding who can view or interact with their content. The system also allows users to block or unblock other users when needed.

9 Key Insights: Facebook

Great work advancing your Facebook system design! This lesson offers practical insights to help you fine-tune your approach, avoid common pitfalls, and confidently tackle interview questions. You 'll also find a short quiz to test your knowledge and related case studies to deepen your practice.

9.1 Common mistakes

While designing Facebook in an interview, many candidates face similar pitfalls that weaken their solution or presentation.

- **Ignoring privacy details:** Candidates often overlook designing how privacy settings work for profiles, posts, groups, and pages. For example, they may forget to clarify how friends-only or custom visibility impacts post delivery and notifications. Always design and explain a dedicated class with clear methods for setting visibility. Show how these settings control what friends, followers, or custom lists can see.
- **Weak search design:** Many candidates skip explaining how the `SearchCatalog` efficiently handles searching for users, posts, pages, and groups, or how the `Search` interface should be implemented to avoid redundant logic. Keep search logic in a separate `SearchCatalog` class that implements a `Search` interface. Clarify that it indexes users, posts, pages, and groups efficiently, avoiding search logic in the `User` or `Page` classes.

- **Neglecting friend request flow:** Some candidates forget to design the full friend request life cycle, i.e., from sending, accepting/rejecting, to status updates. They skip the `FriendRequest` class or fail to show its states (PENDING, ACCEPTED, REJECTED, CANCELED). This weakens the design for user connections. Always include a dedicated `FriendRequest` class with clear methods (`sendFriendRequest()`, `acceptRequest()`, `rejectRequest()`), and link it to notifications for status changes.

- **Missing observer pattern for notifications:** Some fail to identify that Facebook's notification mechanism, especially for groups and posts, can be well modeled with the Observer pattern to keep subscribers updated dynamically. Explain that groups and posts use the Observer pattern, i.e., members (subscribers) register for updates, and `Notification` handles notify them whenever an event occurs. Also, mention unsubscribe options.

- **Overlooking messaging flows:** Candidates sometimes ignore the need for clear message handling. Facebook's design should separate `Message` as its class with methods to add recipients, handle content, and trigger notifications rather than mixing it with unrelated user actions.

9.2 Key interview tips

These actionable tips help you communicate your Facebook design more effectively.

Tip	**Explanation**
Design using interface-based modularity.	Use interfaces like PostFunctionsByUser, Search, and GroupFunctionsByUser to decouple behavior and enable scalability.
Emphasize relationship modeling.	Showcase generalization (e.g., User/Admin), composition (e.g., Profile → Education), and aggregation to reflect real-world object interactions.
Highlight the news feed generation strategy.	Discuss how posts from friends, pages, and groups are aggregated and ordered using feed ranking, time decay, and relevance, showing real-world system thinking.
Discuss access control and privacy.	Model content visibility using classes like PostPrivacySettings and ProfilePrivacy, and explain how different access levels affect the UI/UX logic.
Highlight how notifications are linked to actions.	Explain how notifications tie into key actions like sending friend requests, comments, and posts. For example, when a user sends a friend request, show how the FriendRequest class triggers a notification. This demonstrates understanding of "include" relationships and user engagement.

Table 1.3: Design and Explanation Tips for Social Media System

9.3 Test your understanding

Check how well you've grasped the Facebook system design by tackling these tricky questions that test your OOD thinking and understanding of the design approach.

Facebook

Question 1: Which class best demonstrates the Single Responsibility principle when handling user profile details?

- `User`
- `Account`
- **[CORRECT]** `Profile`
  Explanation: The `Profile` class focuses solely on personal info, keeping the `User` class clean for actions.
- `Admin`

Question 2: Which relationship best describes the link between `User` and `Admin`?

- Association
- **[CORRECT]** Inheritance
  Explanation: Admin generalizes User.
- Aggregation
- Composition

Question 3: Suppose we want to add a feature that lets users undo deleting a comment. Which design pattern would be the best fit for this?

- **[CORRECT]** Command pattern
  Explanation: The Command pattern turns actions into objects. This makes it possible to store, queue, and undo actions like deleting a comment.
- State pattern
- Template Method pattern
- Visitor pattern

Question 4: What is the relationship between `Profile` and `Work`, `Education`, and `Places`?

- Composition
- Inheritance
- Generalization
- **[CORRECT]** Aggregation
  Explanation: They're aggregated into `Profile`.

Question 5: In the friend request flow, which relationship ensures that sending a request always triggers a notification?

- Generalization relationship
- Association relationship

- **[CORRECT]** Include relationship

  Explanation: The include relationship shows that the Send Friend Request use case always includes Send Friend Request Notification.

- Aggregation relationship

9.4 Related case studies

These related case studies build on similar OOD challenges you tackled with Facebook.

- **Designing Stack Overflow:** Highlights user profiles, posts, comments, votes, and notifications, reinforcing content interaction and discoverability.
- **Designing Amazon's online shopping system:** This section explores user accounts, privacy for orders, personalized recommendations, and notifications, a good contrast to Facebook's social feed.
- **Designing LinkedIn:** Adds professional connections, recommendations, and groups with more nuanced privacy and profile structures.
- **Designing Twitter:** Similar focus on feed generation, follower relationships, and notifications with a different interaction model (following vs. friending).
- **Designing Instagram:** This also involves posts, comments, likes, privacy, and notification flows, especially visual content and stories.